COMMONWEALTH OF AUSTRALIA
Copyright Regulations 1969
This material has been reproduced and communicated to you by or on behalf of Monash University pursuant to Part VB of the Copyright Act 1968 (the Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice

Prepared by: [Arun Konagurthu]
Extended by: Taylor Kearney

FIT3155: Advanced Algorithms and Data Structures

Week 2: **Linear-time string pattern matching (Boyer-Moore, Knuth-Morris-Pratt algorithms)**

Faculty of Information Technology, Monash University

Last week

We looked at...

The Z-algorithm

The Z-algorithm is a general (preprocessing) algorithm to Computes Z_i-values of a string **str** in $\mathcal{O}(|\mathbf{str}|)$-time (refer week 1's lecture).

The exact pattern matching problem, addressed using Z-algorithm

Given a reference text **txt**$[1 \ldots n]$ and a pattern **pat**$[1 \ldots m]$, we used the Z-algorithm to find **ALL** occurrences, if any, of **pat** in **txt** in $\mathcal{O}(m + n)$ time.

Week 1 recap: Naïve approach

Example

	1	2	3	4	5	6	7	8	9	10	11	12	13
txt:	x	a	b	x	y	a	b	x	y	a	b	x	z
pat:	a	b	x	y	a	b	x	z					
iteration-1	✗												
pat:		a	b	x	y	a	b	x	z				
iteration-2		✓	✓	✓	✓	✓	✓	✓	✗				
pat:			a	b	x	y	a	b	x	z			
iteration-3			✗										
pat:				a	b	x	y	a	b	x	z		
iteration-4				✗									
pat:					a	b	x	y	a	b	x	z	
iteration-5					✗								
pat:						a	b	x	y	a	b	x	z
iteration-6						✓	✓	✓	✓	✓	✓	✓	✓

Overall 20 comparisons in the naïve approach, on this example.

Week 1 recap: Main ideas to improve naïve pattern matching

We imagined ideas to:

Shift more purposefully: Shift **pat** by > 1 position after each iteration, whenever possible.

Shift always safely: Never shift so far as to miss any exact occurrence of **pat** in **txt**.

Week 1 recap: Main ideas to improve naïve pattern matching

We imagined ideas to:

Shift more purposefully: Shift **pat** by > 1 position after each iteration, whenever possible.

Shift always safely: Never shift so far as to miss any exact occurrence of **pat** in **txt**.

Two specific algorithms (considered classics in computer science) implement these ideas efficiently and in concrete ways:

1. Boyer-Moore
2. Knuth-Morris-Pratt

The Boyer-Moore Algorithm

Comments:

- Defines the standard benchmark for string pattern matching.
- Many find/search utilities that come packaged within programming languages/operating systems rely on this algorithm.
- Incorporated within GNU's implementation of **`grep`**.

Boyer-Moore Algorithm – Introduction

Boyer-Moore algorithm incorporate three main ideas:

1. right-to-left scanning
2. bad-character shift rule
3. good-suffix shift rule
   - matched-prefix shift rule (a variant of the good-suffix rule used in special scenarios)

We will discuss each of these in detail!

Boyer-Moore: General structure of the algorithm

- **`Preprocessing phase`** (performed once at the start of the algorithm solely using **`pat`**):
  - Preprocess **`pat`** to compute three data structures to support
    1. shifting under bad-character shift rule.
    2. shifting under good-suffix shift rule.
    3. shifting under matched-prefix shift rule.

       matched-prefix shifting is used in any iteration when (i) either, another instance of good suffix is not found in the **`pat`**, (ii) or after **`pat`** matches the entire region in the **`txt`**.
- **`Iterative search phase`** (i.e. iterated until end of **`txt`**):
  - In each iteration where **`pat`** is under some region of **`txt`**, compare/scan **`pat`** and **`txt`** characters of that region from right to left.
  - Shift **`pat`** rightwards safely* under **`txt`** by the maximum number of positions allowed by:
    - ★ either bad-character shift rule
    - ★ or good-suffix/matched-prefix shift rule.

*A shift is safe if and only if such a shift does not skip past (and miss) any exact occurrence of **`pat`** in **`txt`**.

Right-to-left scanning

For any comparison of **pat**$[1 \ldots m]$ against **txt**$[j \ldots j+m-1]$, the Boyer-Moore algorithm checks/scans for matched characters right to left.

Example: right to left scanning (in some arbitrary iteration)

```
     1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
txt: y p b c t b y a b p  q  y  c  t  b  p  q  p
                   ←
         1 2 3 4 5 6 7
pat:     t b a p y a b
```

Scanning from right to left in the above example:

- Compare **pat**$[7] \equiv$ b with **txt**$[9] \equiv$ b: match.
- Compare **pat**$[6] \equiv$ a with **txt**$[8] \equiv$ a: match.
- Compare **pat**$[5] \equiv$ y with **txt**$[7] \equiv$ y: match.
- Compare **pat**$[4] \equiv$ p with **txt**$[6] \equiv$ b: mismatch.

After encountering a mismatch in any right-to-left scanning, Boyer-Moore uses information from the character pairs already compared in the current iteration to determine the maximum **safe** rightward shift (by ≥ 1 **positions**) of the pattern **pat** under the text **txt**. It employs two shift rules to do this: (1) the bad-character shift rule and (2) the good-suffix shift rule.

Preprocessing 'Bad-character' shift rule data structure

- Let **pat**$[1 \ldots m]$ and **txt**$[1 \ldots n]$ be strings from the alphabet Σ.
- Preprocess **pat**, and store for each character $x \in \Sigma$, the **rightmost** position of occurrence of character x in **pat**.
  - Call this position, $R(x)$.
  - When x does not occur in **pat**, $R(x) = 0$ (in 1-based indexing of **pat**).

$R(x)$ values stored in an array data structure

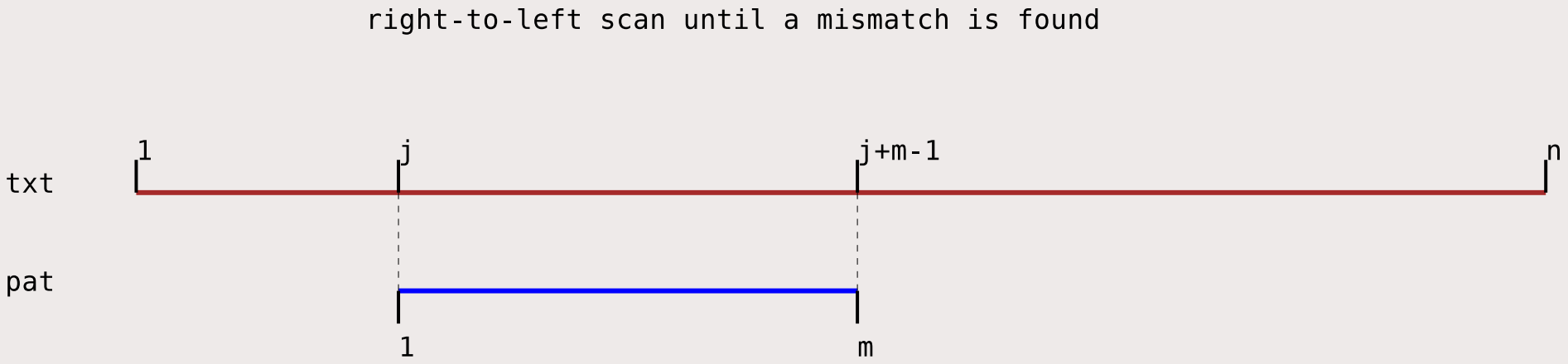
The $R(x)$ values for **pat**$[1 \ldots 7] =$ $\overset{1}{\mathtt{t}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{p}}\ \overset{5}{\mathtt{y}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\mathtt{b}}$ are:

$\forall x \in \Sigma$	a	b	c	...	p	q	...	t	...	y	...
$R(x)$	6	7	0	...	4	0	...	1	...	5	...

Complexity

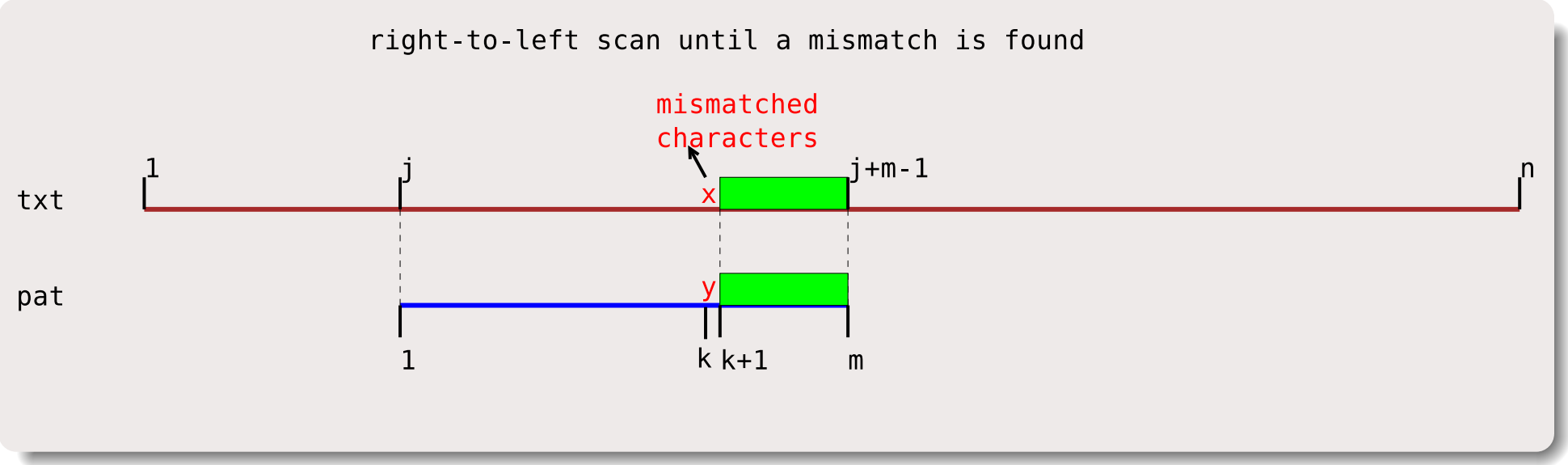
Precomputing $R(x)$ values for any given **pat** requires $\mathcal{O}(|\Sigma|)$ space and $\mathcal{O}(m + |\Sigma|)$ time. After computing this array, during any iteration of Boyer-Moore's **txt** searching phase, it takes $\mathcal{O}(1)$-time to lookup this array and get the rightmost position of any possible (bad-)character in **txt** and derive how many positions to shift **pat** by rightwards to start the next iteration.

Formalizing 'Bad character' shift rule



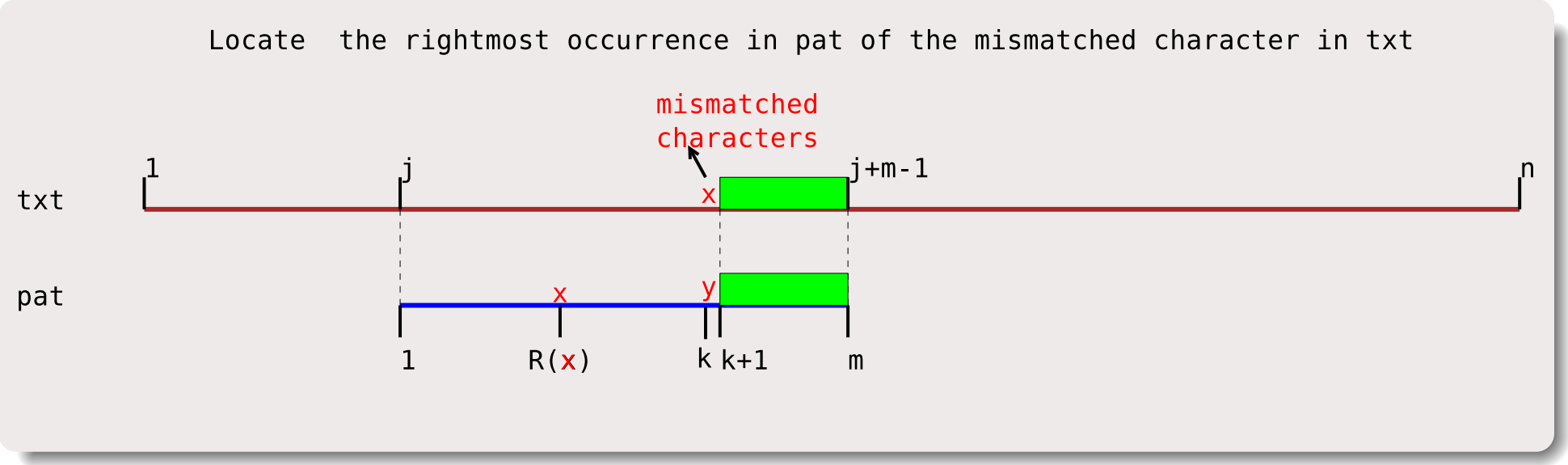
- More abstractly, in some iteration, if **txt**$[j \ldots j+m-1]$ and **pat**$[1 \ldots m]$ are being compared via the right-to-left scan, and

Formalizing 'Bad character' shift rule



- More abstractly, in some iteration, if **txt**$[j \ldots j+m-1]$ and **pat**$[1 \ldots m]$ are being compared via the right-to-left scan, and
- ...the kth character of **txt** counting from position j, i.e. $x =$ **txt**$[j+k-1]$, be the bad character that mismatched with the kth character of the pattern **y** $=$ **pat**$[k]$, then

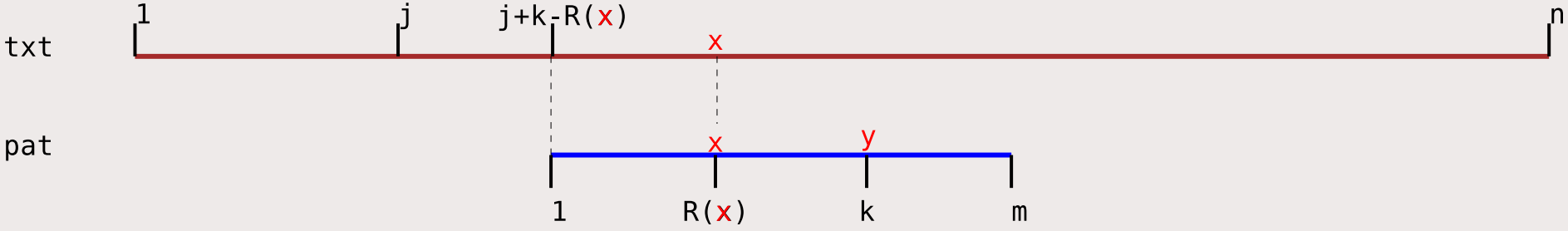
Formalizing 'Bad character' shift rule



- More abstractly, in some iteration, if **txt**$[j \ldots j+m-1]$ and **pat**$[1 \ldots m]$ are being compared via the right-to-left scan, and
- ...the kth character of **txt** counting from position j, i.e. $x =$ **txt**$[j+k-1]$, be the bad character that mismatched with the kth character of the pattern **y** $=$ **pat**$[k]$, then
- ...using the preprocessed $R(x)$ data structure, (potentially) locate the rightmost occurrence $R(x)$ in **pat** of the bad character x.

Formalizing 'Bad character' shift rule

```
Shift pat so that the rightmost occurrence in pat of the mismatched character in txt
will aligned after the shift
```

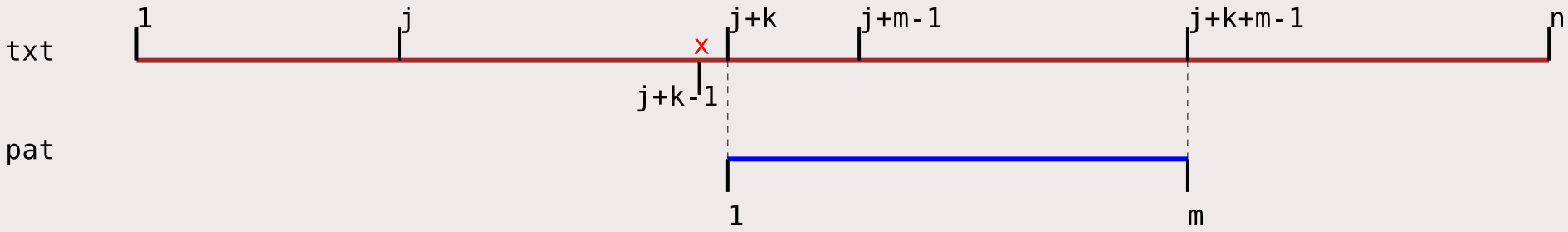


- More abstractly, in some iteration, if **txt**$[j \ldots j+m-1]$ and **pat**$[1 \ldots m]$ are being compared via the right-to-left scan, and
- ...the kth character of **txt** counting from position j, i.e. $x =$ **txt**$[j+k-1]$, be the bad character that mismatched with the kth character of the pattern **y** $=$ **pat**$[k]$, then
- ...using the preprocessed $R(x)$ data structure, (potentially) locate the rightmost occurrence $R(x)$ in **pat** of the bad character x.

Bad-character shift RULE: If $R(x) < k$, shift **pat** rightwards by $k - R(x)$ positions and start the next iteration. Else, one has no choice with this basic array data structure but to shift **pat** rightwards naïvely by 1 position.

Formalizing 'Bad character' shift rule

```
Assume the bad character (x) in text were NOT found in pat. This implies R(x) = 0.
The rule automatically implies that the shift of pat goes past the mismatched
position in txt.
```



- More abstractly, in some iteration, if **txt**$[j \ldots j+m-1]$ and **pat**$[1 \ldots m]$ are being compared via the right-to-left scan, and
- ...the kth character of **txt** counting from position j, i.e. $x =$ **txt**$[j+k-1]$, be the bad character that mismatched with the kth character of the pattern **y** $=$ **pat**$[k]$, then
- ...using the preprocessed $R(x)$ data structure, (potentially) locate the rightmost occurrence $R(x)$ in **pat** of the bad character x.

Bad-character shift RULE: If $R(x) < k$, shift **pat** rightwards by $k - R(x)$ positions and start the next iteration. Else, one has no choice with this basic array data structure but to shift **pat** rightwards naïvely by 1 position.

- Note: The above rule automatically implies that if x does not occur in **pat**$[1..m]$ (i.e., $R(x) = 0$), then the **pat** gets shifted one position past the point of mismatch in **txt**.

Bad character shift rule example

Consider the following pattter and text strings:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
txt:	y	p	b	c	t	b	y	a	b	a	c	b	y	t	b	p	q	p
	1	2	3	4	5	6	7											
pat:	t	b	a	p	y	a	b											

Note: **pat** string slightly different from previous.

The $R(x)$ values for the pattern are:

$\forall x \in \Sigma$	a	b	c	...	p	q	...	t	...	y	...
$R(x)$	6	7	0	...	4	0	...	1	...	5	...

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
txt:	y	p	b	c	t	b	y	a	b	a	c	b	y	t	b	p	q	p
iteration 1							✗											
pat:	t	b	a	p	y	a	b											
	1	2	3	4	5	6	7											

Shift by $\max(1, k - R(x)) = \max(1, 7 - 5) = 2$ positions to start next iteration.

Bad character shift rule example

Consider the following pattter and text strings:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
txt:	y	p	b	c	t	b	y	a	b	a	c	b	y	t	b	p	q	p
	1	2	3	4	5	6	7											
pat:	t	b	a	p	y	a	b											

Note: **pat** string slightly different from previous.

The $R(x)$ values for the pattern are:

$\forall x \in \Sigma$	a	b	c	...	p	q	...	t	...	y	...
$R(x)$	6	7	0	...	4	0	...	1	...	5	...

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
txt:	y	p	b	c	t	b	y	a	b	a	c	b	y	t	b	p	q	p
iteration 2						✗	✓	✓	✓									
pat:			t	b	a	p	y	a	b									
			1	2	3	4	5	6	7									

Shift by $\max(1, k - R(x)) = \max(1, 4 - 7) = 1$ position to start next iteration.

Bad character shift rule example

Consider the following pattter and text strings:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
txt:	y	p	b	c	t	b	y	a	b	a	c	b	y	t	b	p	q	p
	1	2	3	4	5	6	7											
pat:	t	b	a	p	y	a	b											

Note: **pat** string slightly different from previous.

The $R(x)$ values for the pattern are:

$\forall x \in \Sigma$	a	b	c	...	p	q	...	t	...	y	...
$R(x)$	6	7	0	...	4	0	...	1	...	5	...

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
txt:	y	p	b	c	t	b	y	a	b	a	c	b	y	t	b	p	q	p
iteration 3										✗								
pat:				t	b	a	p	y	a	b								
				1	2	3	4	5	6	7								

Shift by $\max(1, k - R(x)) = \max(1, 7 - 6) = 1$ position to start next iteration.

Bad character shift rule example

Consider the following pattter and text strings:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
txt:	y	p	b	c	t	b	y	a	b	a	c	b	y	t	b	p	q	p
	1	2	3	4	5	6	7											
pat:	t	b	a	p	y	a	b											

Note: **pat** string slightly different from previous.

The $R(x)$ values for the pattern are:

$\forall x \in \Sigma$	a	b	c	...	p	q	...	t	...	y	...
$R(x)$	6	7	0	...	4	0	...	1	...	5	...

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
txt:	y	p	b	c	t	b	y	a	b	a	c	b	y	t	b	p	q	p
iteration 4											✗							
pat:					t	b	a	p	y	a	b							
					1	2	3	4	5	6	7							

Shift by $\max(1, k - R(x)) = \max(1, 7 - 0) = 7$ positions to start next iteration.

Bad character shift rule example

Consider the following pattter and text strings:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
txt:	y	p	b	c	t	b	y	a	b	a	c	b	y	t	b	p	q	p
	1	2	3	4	5	6	7											
pat:	t	b	a	p	y	a	b											

Note: **pat** string slightly different from previous.

The $R(x)$ values for the pattern are:

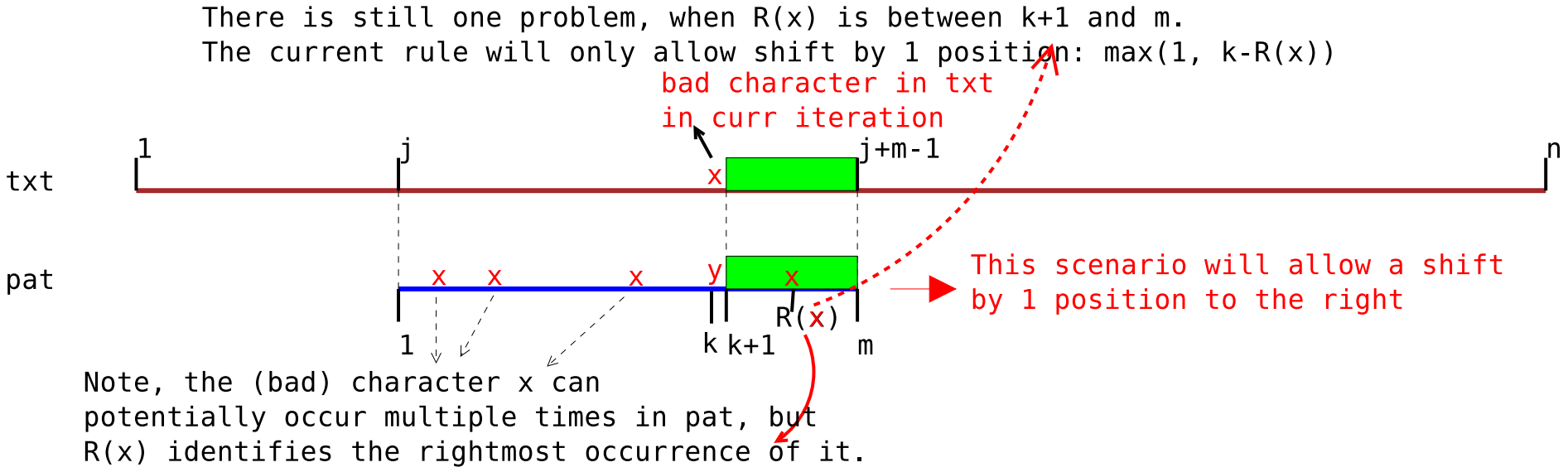
$\forall x \in \Sigma$	a	b	c	...	p	q	...	t	...	y	...
$R(x)$	6	7	0	...	4	0	...	1	...	5	...

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
txt:	y	p	b	c	t	b	y	a	b	a	c	b	y	t	b	p	q	p
iteration 5																		✗
pat:												t	b	a	p	y	a	b
												1	2	3	4	5	6	7

Shift by $\max(1, k - R(x)) = \max(1, 7 - 4) = 3$ positions to start next iteration (if more text still available; otherwise stop)

Improving further: The extended bad-character shift rule

Recall from the bad character shift rule (see slide 11, last overlay), that when $R(x) > k$, then we shift naïvely by 1 position.

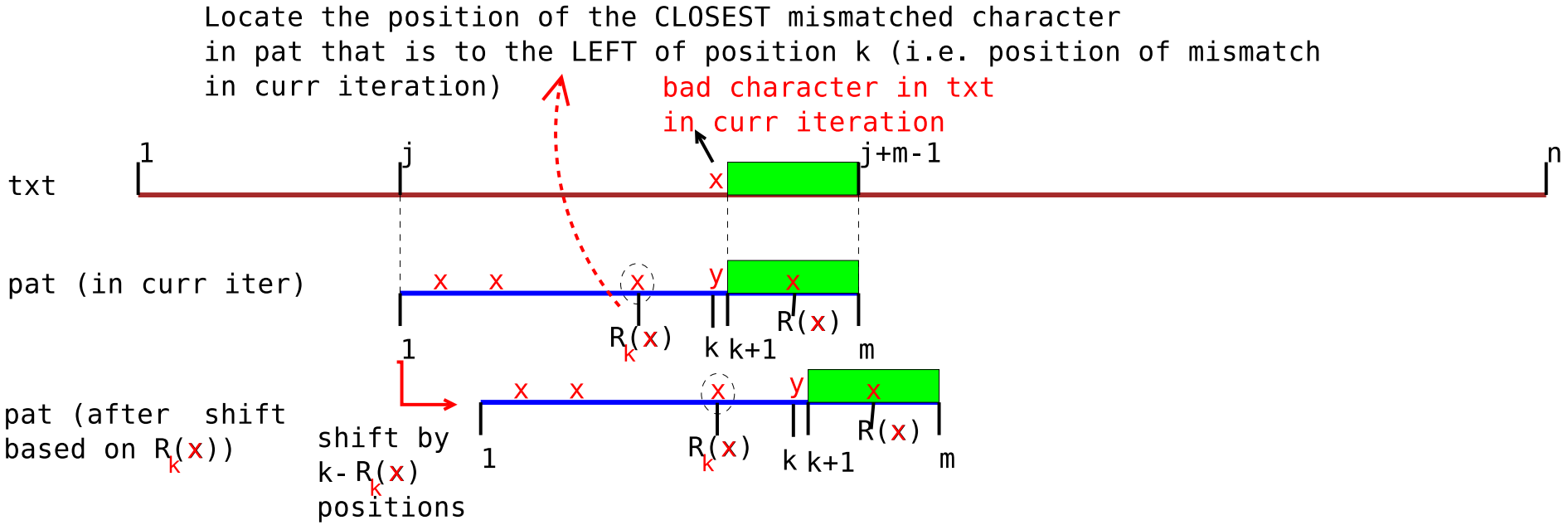


How can we extend the $R(\cdot)$ data structure so that we can do better than that, when any $R(x) > k$?

Improving further: The extended bad-character shift rule

Extended Bad-Character Rule

When a mismatch occurs at some position k in **pat**$[1 \ldots m]$, and the corresponding mismatched character is $x =$ **txt**$[j + k - 1]$, then **shift** **pat**$[1..m]$ to the right so that **the closest x in pat that is to the left of position** k is now below the (previously mismatched) x in **txt**.



Improving further: The extended bad-character shift rule

Preprocessing

- To achieve this, preprocess **pat**$[1 \ldots m]$ to compute $R_k(x)$ values...
  - for each position $1 \leq k \leq m$ in **pat**,
  - and for each character x in the alphabet Σ,

  ...storing the **rightmost** position of x **to the left of** $(<)$ k

In Boyer-Moore's search phase...

- For any possible combinations of k and x, if we denote such a position as $R_k(x)$, then this data strcuture implementing the extended bad character shift rule will ask you to shift **pat safely** by $k - R_k(x)$ positions, when a mismatch occurs at any position k in **pat** with x as the bad-character in the **txt**.

Complexity

The most basic implementation of the $R_k(x)$ data structure is a 2D array/table, taking $\mathcal{O}(m \times |\Sigma|)$ space and time to precompute (example in next slide). In turn, this will provide $\mathcal{O}(1)$ lookup during the search phase.

(Quiz: Can this is implemented in $\mathcal{O}(m+|\Sigma|)$ space? Does this trade-off lookup time?)

Improving further: The extended bad-character shift rule

Self-study: Example of a basic extended $R_k(\cdot)$ data structure

For **pat**$[1 \ldots 7] =$ $\overset{1}{\mathtt{t}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{p}}\ \overset{5}{\mathtt{x}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\mathtt{b}}$, one could store the corresponding $R_k(x)$ values in the following rudimentary 2D matrix of size $m \times |\Sigma|$:

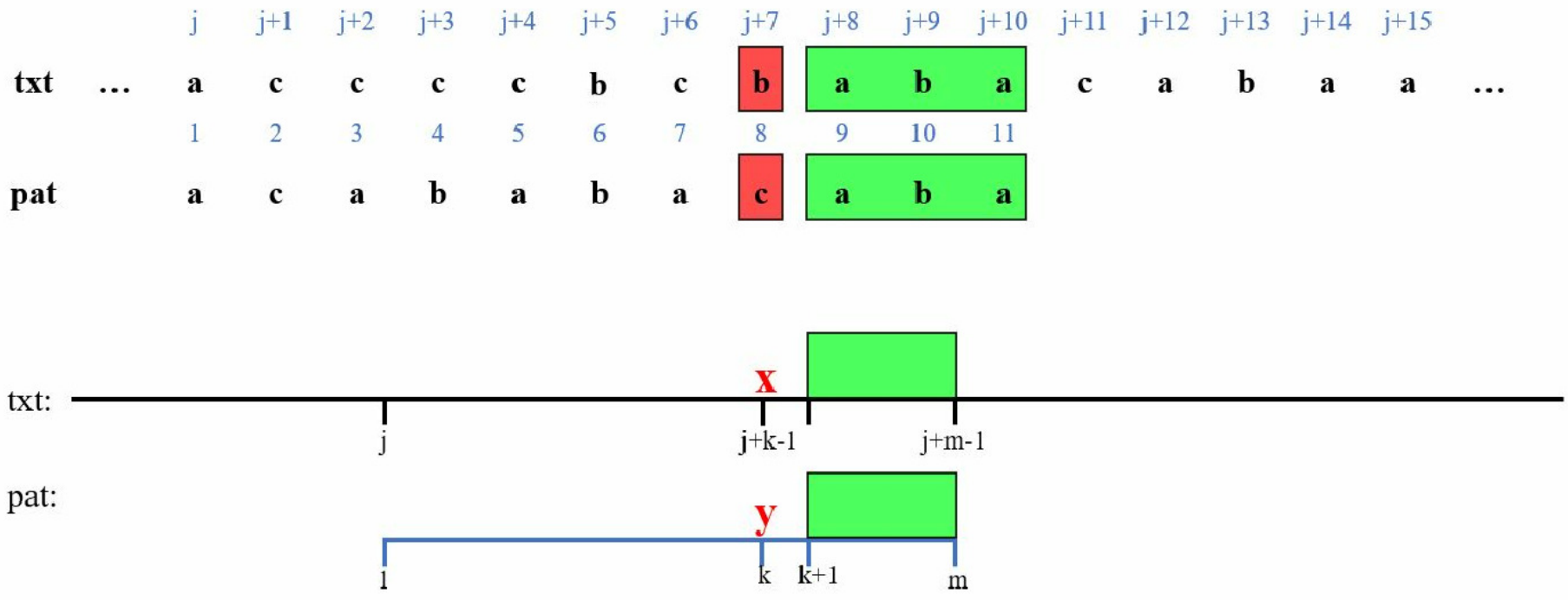
$\forall x \in \Sigma$	a	b	c	...	p	q	...	t	...	x	...
$R_1(x)$	0	0	0	...	0	0	...	0	...	0	...
$R_2(x)$	0	0	0	...	0	0	...	1	...	0	...
$R_3(x)$	0	2	0	...	0	0	...	1	...	0	...
$R_4(x)$	3	2	0	...	0	0	...	1	...	0	...
$R_5(x)$	3	2	0	...	4	0	...	1	...	0	...
$R_6(x)$	3	2	0	...	4	0	...	1	...	5	...
$R_7(x)$	6	2	0	...	4	0	...	1	...	5	...

Boyer-Moore Algorithm – Good suffix rule: Motivation

		j	j+1	j+2	j+3	j+4	j+5	j+6	j+7	j+8	j+9	j+10	j+11	j+12	j+13	j+14	j+15	
txt	**...**	**a**	**c**	**c**	**c**	**c**	**b**	**c**	**b**	**a**	**b**	**a**	**c**	**a**	**b**	**a**	**a**	**...**
		1	2	3	4	5	6	7	8	9	10	11						
pat		**a**	**c**	**a**	**b**	**a**	**b**	**a**	**c**	**a**	**b**	**a**						

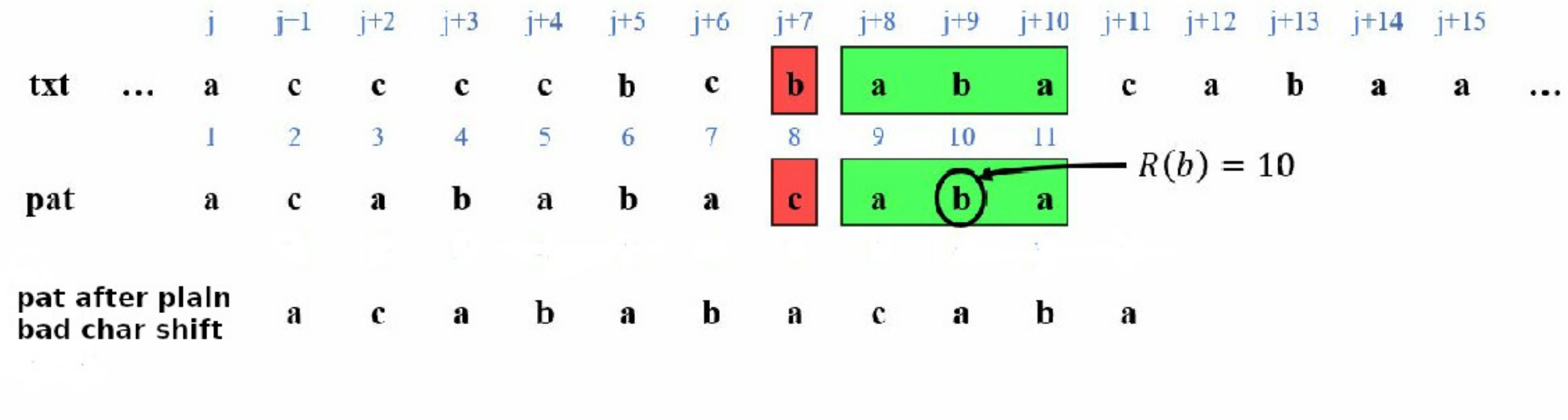
- In some iteration, say **txt**$[j \ldots j+m-1]$ and **pat**$[1 \ldots m]$ are being compared via right-to-left scan.

Boyer-Moore Algorithm – Good suffix rule: Motivation



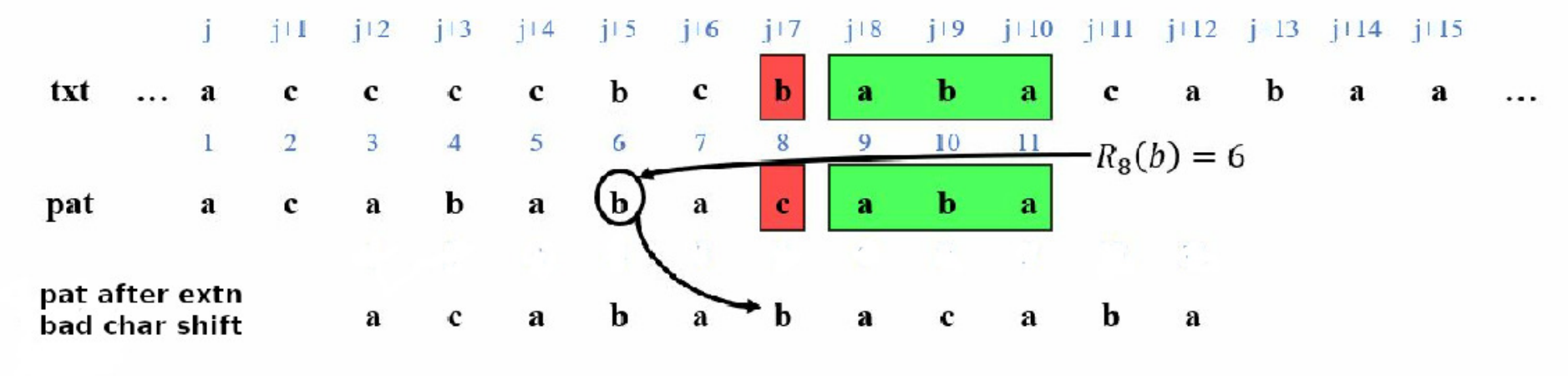
- In some iteration, say **txt**$[j \ldots j+m-1]$ and **pat**$[1 \ldots m]$ are being compared via right-to-left scan.
- Let the kth character of **txt** counting from j, i.e. $x =$ **txt**$[j+k-1]$, be mismatched with the kth character of the pattern **y** $=$ **pat**$[k]$.

Boyer-Moore Algorithm – Good suffix rule: Motivation



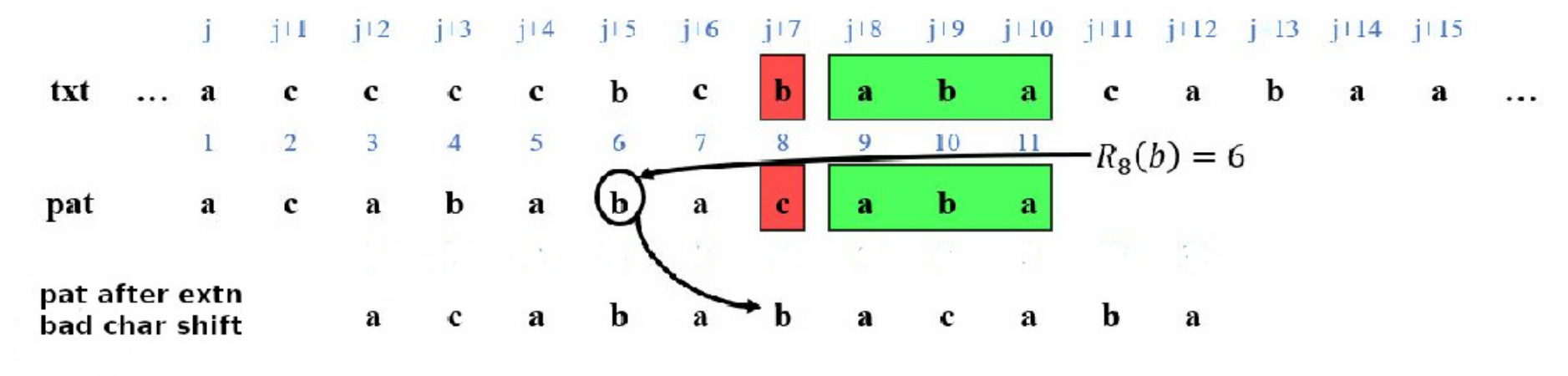
- In some iteration, say $\texttt{txt}[j \ldots j+m-1]$ and $\texttt{pat}[1 \ldots m]$ are being compared via right-to-left scan.
- Let the kth character of **txt** counting from j, i.e. $x = \texttt{txt}[j+k-1]$, be mismatched with the kth character of the pattern $\texttt{y} = \texttt{pat}[k]$.
- The plain $R(\cdot)$ based bad character rule will only suggest a naïve shift of **pat** by 1 position to the right.

Boyer-Moore Algorithm – Good suffix rule: Motivation



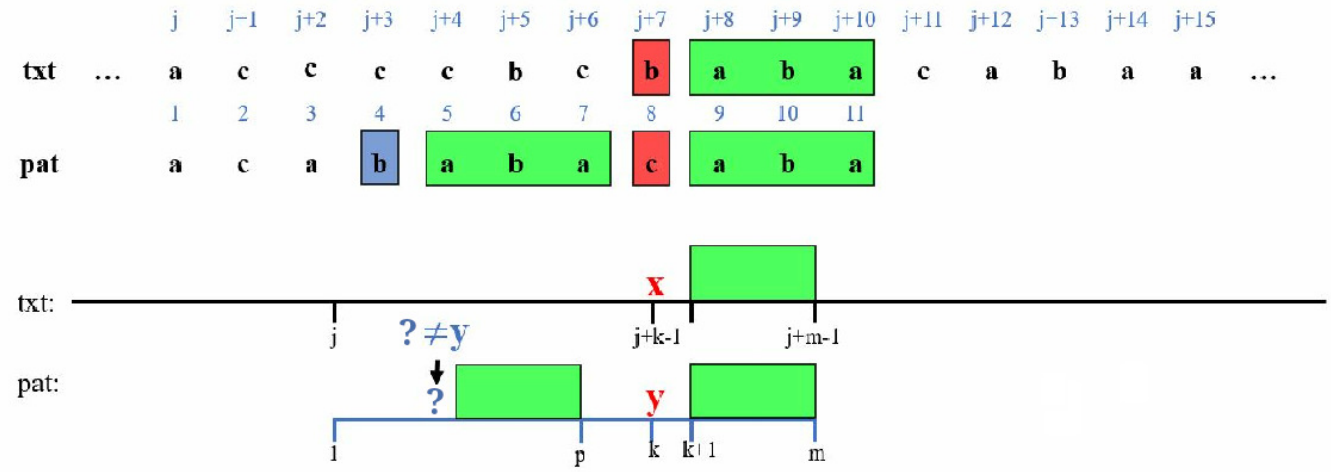
- In some iteration, say **txt**$[j \ldots j+m-1]$ and **pat**$[1 \ldots m]$ are being compared via right-to-left scan.
- Let the kth character of **txt** counting from j, i.e. $x =$ **txt**$[j+k-1]$, be mismatched with the kth character of the pattern **y** $=$ **pat**$[k]$.
- The plain $R(\cdot)$ based bad character rule will only suggest a naïve shift of **pat** by 1 position to the right.
- The extended $R_k(\cdot)$ based bad character rule suggests a shift of 2 positions to the right.

Boyer-Moore Algorithm – Good suffix rule: Motivation



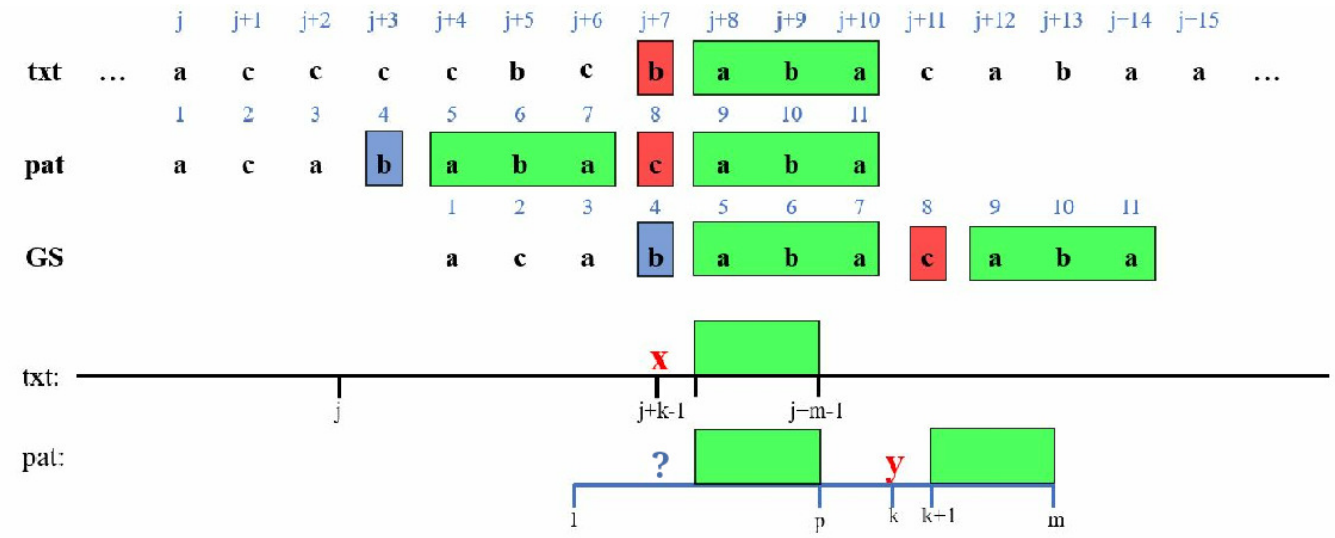
- In some iteration, say **txt**$[j \ldots j+m-1]$ and **pat**$[1 \ldots m]$ are being compared via right-to-left scan.
- Let the kth character of **txt** counting from j, i.e. $x =$ **txt**$[j+k-1]$, be mismatched with the kth character of the pattern **y** $=$ **pat**$[k]$.
- The plain $R(\cdot)$ based bad character rule will only suggest a naïve shift of **pat** by 1 position to the right.
- The extended $R_k(\cdot)$ based bad character rule suggests a shift of 2 positions to the right.
- The motivation behind Good suffix rule is to explore whether the pattern can **safely** be shifted by more positions than what the extended bad character rule suggests.

Boyer-Moore Algorithm – Good suffix rule: Intuition



- Suppose there exists a position $p < m$ in **pat** such that it is the **endpoint** of the rightmost occurrence of the **substring** such that:
  - $\texttt{pat}[p - m + k + 1 \ldots p] \equiv \texttt{pat}[k + 1 \ldots m]$, and
  - $\texttt{pat}[p - m + k] \neq \texttt{pat}[k]$,

Boyer-Moore Algorithm – Good suffix rule: Intuition



- Suppose there exists a position $p < m$ in **pat** such that it is the **endpoint** of the rightmost occurrence of the **substring** such that:
  - $\texttt{pat}[p - m + k + 1 \ldots p] \equiv \texttt{pat}[k + 1 \ldots m]$, and
  - $\texttt{pat}[p - m + k] \neq \texttt{pat}[k]$,
- then **pat** can be safely and efficiently shifted right under the **txt** by $m - p$ positions, to begin a new iteration.

Boyer-Moore Algorithm – Ideas to implement the 'good suffix' rule efficiently

To efficiently implement this 'good suffix' rule, we take 'inspiration' from the computation of Z_i values in Gusfield's algorithm, and define Z_i^{suffix} (specifically on **pat**) as follows:

Boyer-Moore Algorithm – Ideas to implement the 'good suffix' rule efficiently

To efficiently implement this 'good suffix' rule, we take 'inspiration' from the computation of Z_i values in Gusfield's algorithm, and define Z_i^{suffix} (specifically on **pat**) as follows:

Definition of Z_i^{suffix}

Given a **pat**$[1 \ldots m]$, define Z_i^{suffix} (for each position $1 \leq i < m$) as the **length** of the **longest substring ending at position** i of **pat** that matches its suffix (i.e., **pat**[i-Z_i^{suffix}+1 ... i] = **pat**[m-Z_i^{suffix}+1 ... m]).

Boyer-Moore Algorithm – Ideas to implement the 'good suffix' rule efficiently

To efficiently implement this 'good suffix' rule, we take 'inspiration' from the computation of Z_i values in Gusfield's algorithm, and define Z_i^{suffix} (specifically on **pat**) as follows:

Definition of Z_i^{suffix}

Given a **pat**$[1 \ldots m]$, define Z_i^{suffix} (for each position $1 \leq i < m$) as the **length** of the **longest substring ending at position** i of **pat** that matches its suffix (i.e., **pat**[i-Z_i^{suffix}+1 ... i] = **pat**[m-Z_i^{suffix}+1 ... m]).

- Note, computation of Z_i^{suffix} values on **pat** can be derived after running Z-algorithm on reverse(**pat**).
- Thus, Z_i^{suffix} values can be computed in $\mathcal{O}(m)$ time for **pat**$[1 \ldots m]$.

Boyer-Moore Algorithm – Implementing 'good suffix' rule efficiently

In fact, for each **suffix** starting at position $j \in [1 \ldots m]$ in **pat**, we want to store the rightmost position p in **pat** such that:

- $\mathtt{pat}[j..m] \equiv \mathtt{pat}[p - Z_p^{\text{suffix}} + 1 \ldots p]$.
- $\mathtt{pat}[j-1] \neq \mathtt{pat}[p - Z_p^{\text{suffix}}]$.

Boyer-Moore Algorithm – Implementing 'good suffix' rule efficiently

In fact, for each **suffix** starting at position $j \in [1 \ldots m]$ in **pat**, we want to store the rightmost position p in **pat** such that:

- $\mathbf{pat}[j..m] \equiv \mathbf{pat}[p - Z_p^{\text{suffix}} + 1 \ldots p]$.
- $\mathbf{pat}[j-1] \neq \mathbf{pat}[p - Z_p^{\text{suffix}}]$.

Store these rightmost positions as $\mathbf{goodsuffix}(j) = p$.

After computing the Z_p^{suffix} values, $\mathbf{goodsuffix}(j)$ values can be computed from them using this simple logic – reason why during self-study.

```
Z^{suffix} := computeZSuffixValues(pat)
m := |pat|
for j from 1 to m+1  do
    goodsuffix(j) := 0
endfor

for p from 1 to m-1  do
    j := m - Z^{suffix}(p) + 1
    goodsuffix(j) := p
endfor
```

Self-study: Example of good suffix values computation

For **pat**$[1 \ldots 11] =$ $\overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{c}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{b}}\ \overset{5}{\mathtt{a}}\ \overset{6}{\mathtt{b}}\ \overset{7}{\mathtt{a}}\ \overset{8}{\mathtt{c}}\ \overset{9}{\mathtt{a}}\ \overset{10}{\mathtt{b}}\ \overset{11}{\mathtt{a}}$

Z^{suffix} and goodsuffix values after lines 1-5 of the pseudocode

j	1	2	3	4	5	6	7	8	9	10	11	12
pat	a	c	a	b	a	b	a	c	a	b	a	
Z^{suffix}	1	0	1	0	5	0	3	0	1	0	-	-
goodsuffix	0	0	0	0	0	0	0	0	0	0	0	0

```
1  Z^{suffix} := computeZSuffixValues(pat)
2  m := |pat|
3  for j from 1 to m+1  do
4      goodsuffix(j) := 0
5  endfor
6
7  for p from 1 to m-1  do
8      j := m - Z^{suffix}(p) + 1
9      goodsuffix(j) := p
10 endfor
```

Self-study: Example of good suffix values computation

For **pat**$[1 \ldots 11] =$ $\overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{c}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{b}}\ \overset{5}{\mathtt{a}}\ \overset{6}{\mathtt{b}}\ \overset{7}{\mathtt{a}}\ \overset{8}{\mathtt{c}}\ \overset{9}{\mathtt{a}}\ \overset{10}{\mathtt{b}}\ \overset{11}{\mathtt{a}}$

Z^{suffix} and goodsuffix values after lines 1-10 of the pseudocode

j	1	2	3	4	5	6	7	8	9	10	11	12
pat	a	c	a	b	a	b	a	c	a	b	a	
Z^{suffix}	1	0	1	0	5	0	3	0	1	0	-	-
goodsuffix	0	0	0	0	0	0	5	0	7	0	9	10

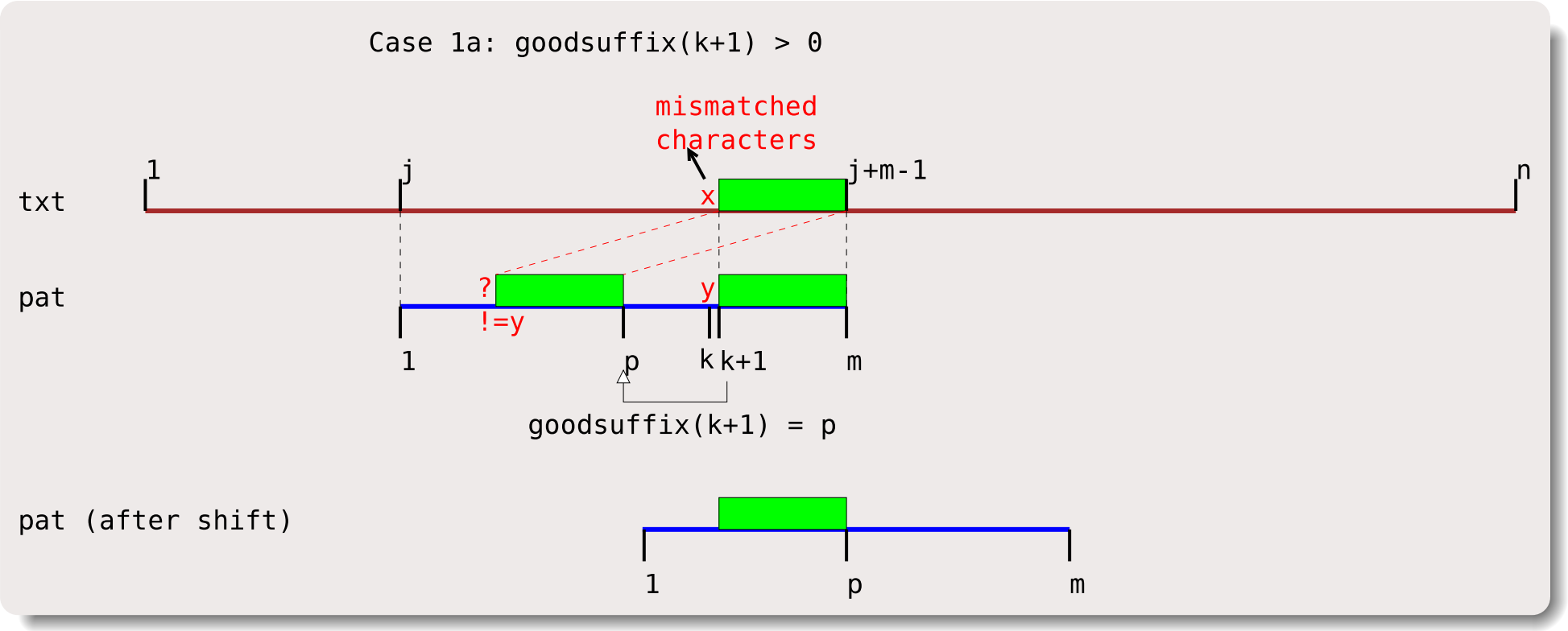
```
1 Z^{suffix} := computeZSuffixValues(pat)
2 m := |pat|
3 for j from 1 to m+1  do
4     goodsuffix(j) := 0
5 endfor
6
7 for p from 1 to m-1  do
8     j := m - Z^{suffix}(p) + 1
9     goodsuffix(j) := p
10 endfor
```

Boyer-Moore Algorithm – Using 'good suffix' rule during search

In any iteration, to use the 'good suffix' rule, the following cases have to handled:

Boyer-Moore Algorithm – Using 'good suffix' rule during search

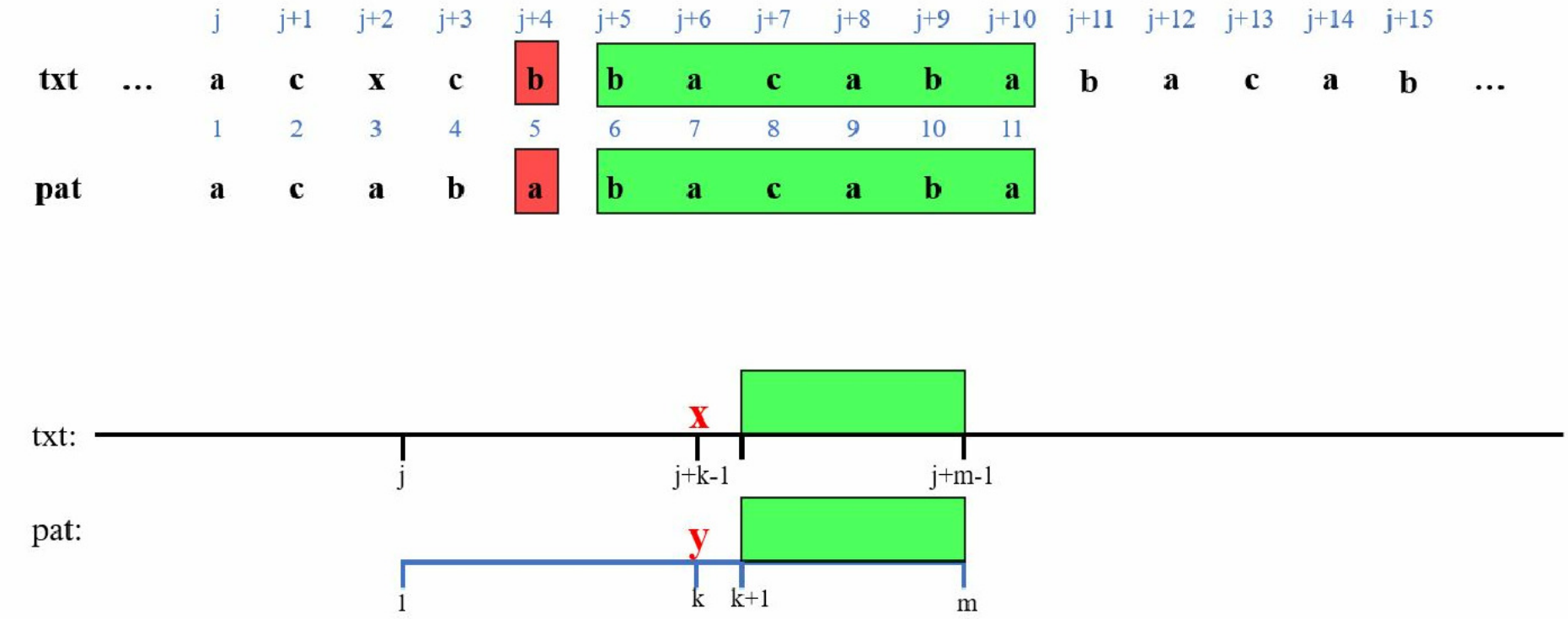
In any iteration, to use the 'good suffix' rule, the following cases have to handled:



Case 1a: if a mismatch occurs at some **pat**$[k]$, and **goodsuffix**$(k+1) > 0$ then

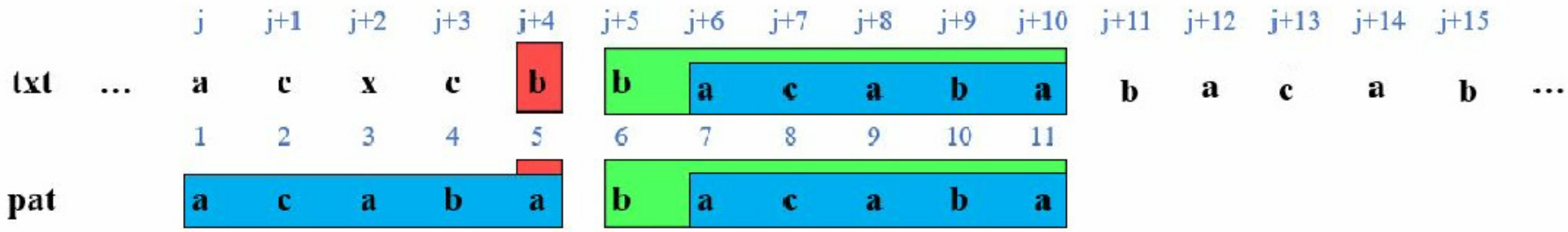
- shift **pat** by $m -$ **goodsuffix**$(k+1)$ places.

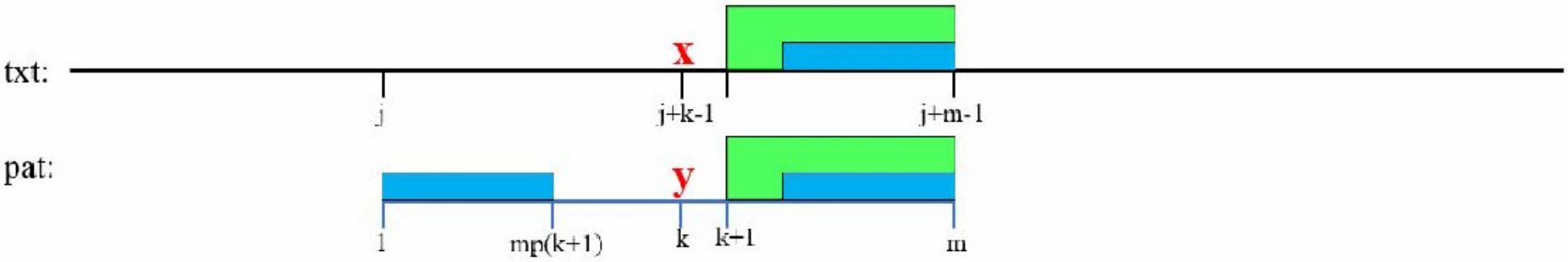
Boyer-Moore Algorithm – Using 'good suffix' rule during search (cont'd)



Case 1b: if a mismatch occurs at some **pat**$[k]$, and **goodsuffix**$(k+1)$=0 then:

Boyer-Moore Algorithm – Using 'good suffix' rule during search (cont'd)

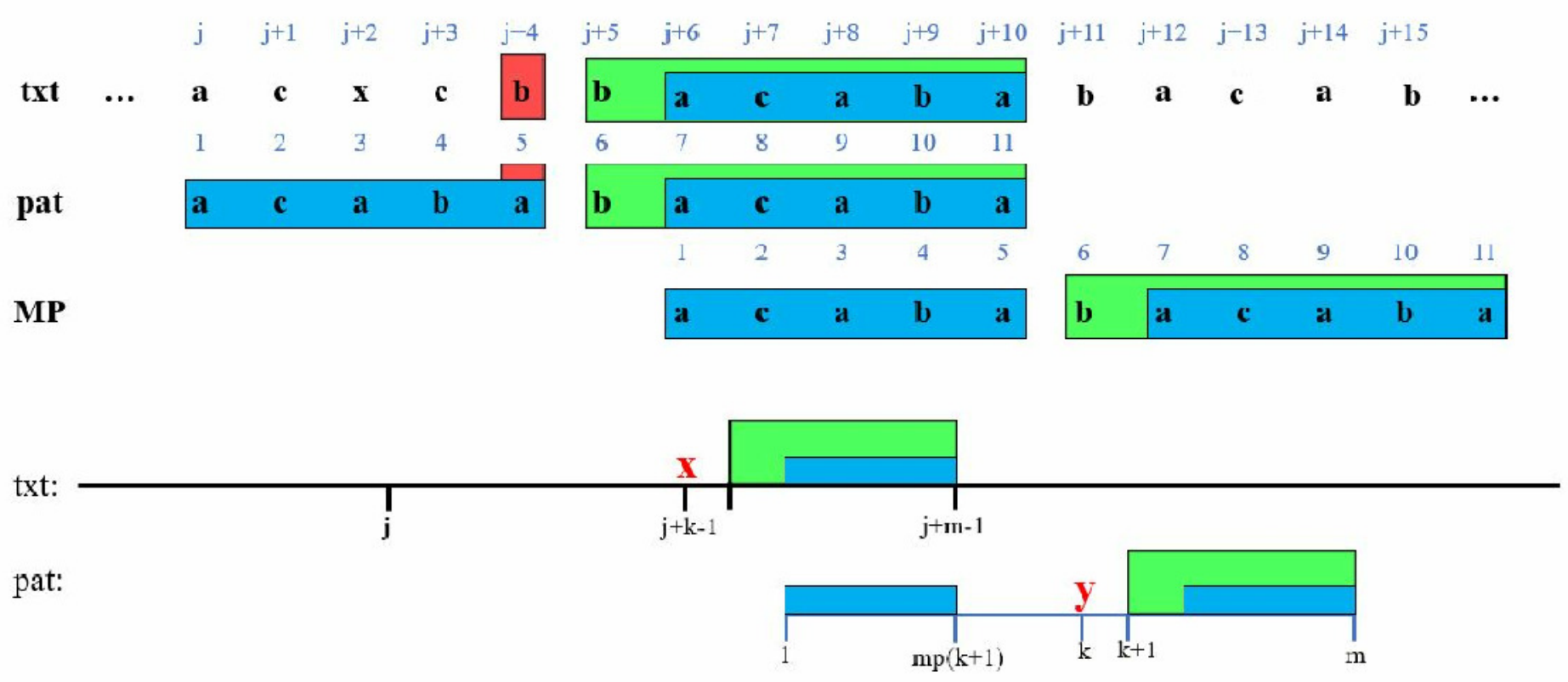




Case 1b: if a mismatch occurs at some **pat**$[k]$, and **goodsuffix**$(k+1)$=0 then:

- **matchedprefix**$(k+1)$ denotes the length of the **largest** suffix of **pat**$[k+1..m]$ that is identical to the prefix of **pat**$[1..m-k]$.
- **matchedprefix**$(.)$ values for **pat** can be precomputed using Z-algorithm in $\mathcal{O}(m)$ time – how?(Hint: z-algorithm – this is a tute question!)

Boyer-Moore Algorithm – Using 'good suffix' rule during search (cont'd)



Case 1b: if a mismatch occurs at some **pat**$[k]$, and **goodsuffix**$(k+1)$=0 then:

- **matchedprefix**$(k+1)$ denotes the length of the **largest** suffix of **pat**$[k+1..m]$ that is identical to the prefix of **pat**$[1..m-k]$.
- **matchedprefix**$(.)$ values for **pat** can be precomputed using Z-algorithm in $\mathcal{O}(m)$ time – how?(Hint: z-algorithm – this is a tute question!)

Shift **pat** by $m -$ **matchedprefix**$(k+1)$ places

Self-study: Example for **matchedprefix** values

Example of computation of **matchedprefix** values

For the same example as before, **pat**$[1 \ldots 11] =$ a c a b a b a c a b a (positions 1 2 3 4 5 6 7 8 9 10 11), see how the **matchedprefix** values compare to Z^{suffix} and goodsuffix.

j	1	2	3	4	5	6	7	8	9	10	11	12
pat	a	c	a	b	a	b	a	c	a	b	a	
Z^{suffix}	1	0	1	0	5	0	3	0	1	0	-	-
goodsuffix	0	0	0	0	0	0	5	0	7	0	9	10
matchedprefix	11	5	5	5	5	5	5	1	1	1	1	0

Boyer-Moore Algorithm – shift rule when pattern is found

```
txt     . . . a a b a a c a b a c a a b a a c a b a c a a . . .
                | | | | | | | | | | | | | |
pat             a a b a a c a b a c a a b a
                1 2 3 4 5 6 7 8 9 10 11 12 13 14
```

Case 2: when **pat**$[1 \ldots m]$ fully matches **txt**$[j \ldots j + m - 1]$

(i.e., the case when an exact occurrence of **pat** was found in **txt** at a position starting from j)

Boyer-Moore Algorithm – shift rule when pattern is found

```
txt     . . . a a b a a c a b a c a a b a a c a b a c a a . . .

pat                                     a a b a a c a b a c a a b a
                                        1 2 3 4 5 6 7 8 9 10 11 12 13 14
```

Case 2: when **pat**$[1 \ldots m]$ fully matches **txt**$[j \ldots j + m - 1]$

(i.e., the case when an exact occurrence of **pat** was found in **txt** at a position starting from j)

- shift **pat** by $m -$ **matchedprefix**(2) places. Why? (Reason what **matchedprefix**(2) tells you about the **pat**.)

Boyer-Moore Algorithm – Bringing all pieces together

Preprocessing step

- Preprocess **pat** . . .
  - . . . for the shift data structure (eg. $R(\cdot)$, or $R_k(\cdot)$ values) needed for 'bad-character' shifts (see slides 11-13)
  - . . . for **goodsuffix**$(\cdot)$ and **matchedprefix**$(\cdot)$ values needed for 'good suffix' shifts (see slides 14-22)

Boyer-Moore Algorithm – Bringing all pieces together

Preprocessing step

- Preprocess **pat** . . .
  - . . . for the shift data structure (eg. $R(\cdot)$, or $R_k(\cdot)$ values) needed for 'bad-character' shifts (see slides 11-13)
  - . . . for **goodsuffix**$(\cdot)$ and **matchedprefix**$(\cdot)$ values needed for 'good suffix' shifts (see slides 14-22)

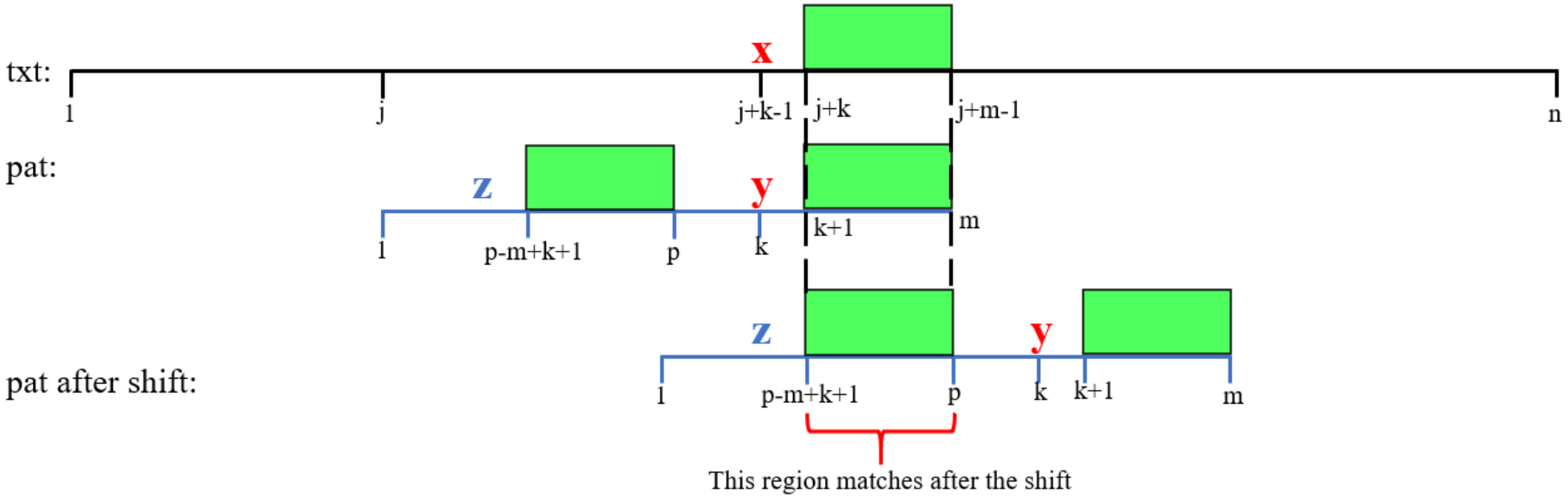
Algorithm

- Starting with **pat**$[1..m]$vs.**txt**$[1..m]$, in each iteration, scan 'right-to-left'
- If, in any iteration, the region matches fully (i.e., **pat** found), shift using case 2 (slide 22)).
- Otherwise (if pattern not found, we have a bad-character and good-suffix):
  - Use (extended) bad-character rule to find how many places to the right **pat** should be shifted under **txt**. Call this shift amount, $s_{\text{badcharacter}}$.
  - Use good suffix rule to find how many places to the right **pat** should be shifted under **txt**. Call this shift amount, $s_{\text{goodsuffix}}$.
  - Shift **pat** to the right under **txt** by $\max(s_{\text{badcharacter}}, s_{\text{goodsuffix}})$ places.

An obvious **optimization** between iterations

Remember!

Between two successive iterations, never recompare character-pairs between **pat** and **txt** that you already know will match. (This certainty forms the very basis of various shifts in this algorithm).



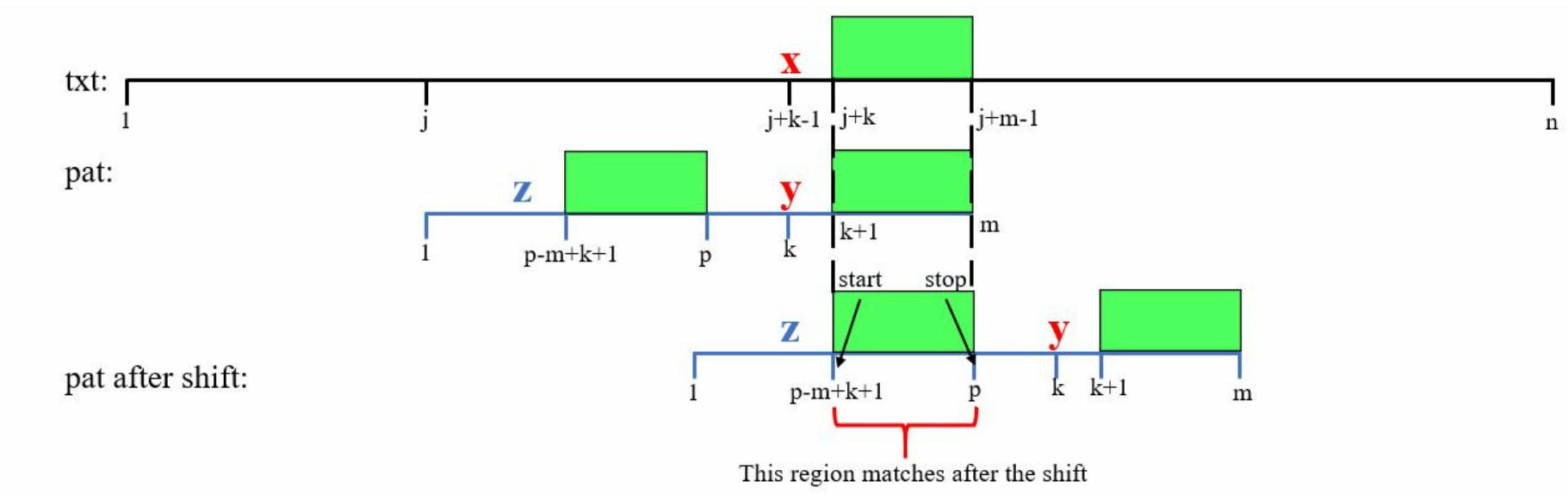
This optimization ensures that **Boyer Moore algorithm** can identify all exact matches of **pat** in **txt** in *worst-case* of $\mathcal{O}(n)$ time, after $\mathcal{O}(m)$ time spent in preprocessing **pat**.

(Non-examinable discussion on time-complexity during the lecture.)

Boyer-Moore Algorithm – Ideas to implement this optimisation

All we need are two variables (say) $start$ and $stop$ to keep track of regions between iterations that are guaranteed to match in the next iteration. For example:

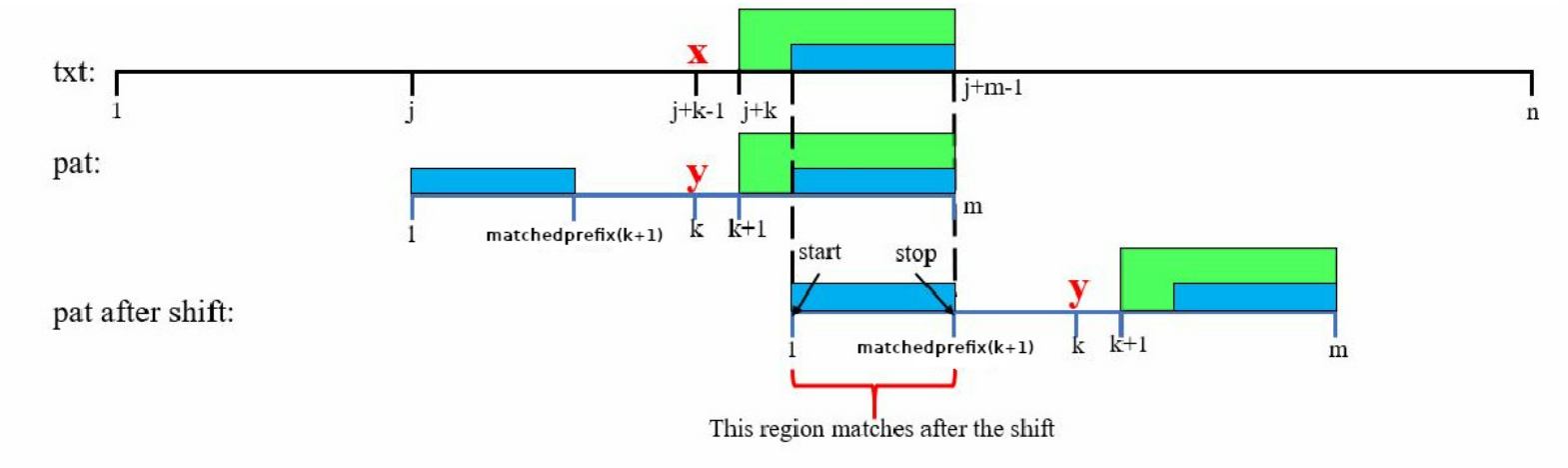
- If **goodsuffix**$(k+1) > 0$, in the lead up to the next iteration, earmark $start =$ **goodsuffix**$(k+1) - m + k + 1$ and $stop =$ **goodsuffix**$(k+1)$.



Boyer-Moore Algorithm – Ideas to implement this optimisation

All we need are two variables (say) $start$ and $stop$ to keep track of regions between iterations that are guaranteed to match in the next iteration. For example:

- If **goodsuffix**$(k+1) > 0$, in the lead up to the next iteration, earmark $start =$ **goodsuffix**$(k+1) - m + k + 1$ and $stop =$ **goodsuffix**$(k+1)$.
- Else, earmark $start = 1$ and $stop =$ **matchedprefix**$(k+1)$.

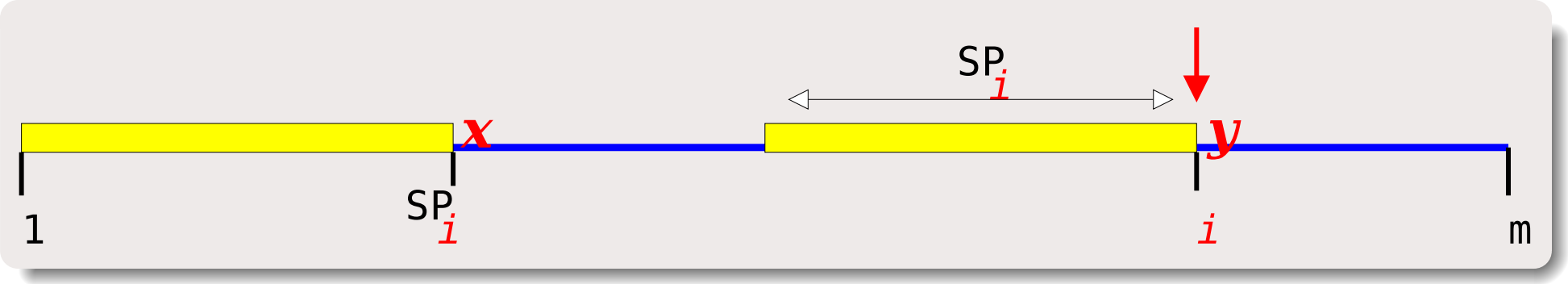


Knuth-Morris-Pratt (KMP) Algorithm

Comments:

- KMP is a very popular algorithm covered in undergrad CS courses.
- Although, it is rarely the method of choice..
- ...and is inferior in performance to Boyer-Moore we discussed above.
- It's explanation is rather simple, as we will see below.
- In fact, much like we did in BM, we will also approach KMP employing the Z-algorithm, which makes the whole implementation even cleaner/better.

KMP algorithm – Defining $\mathbf{SP}_i$ values for $\mathbf{pat}[1 \ldots m]$



Definition of $\mathbf{SP}_i$:

Given a pattern $\mathbf{pat}[1 \ldots m]$, define $\mathbf{SP}_i$ (for each position i in $\mathbf{pat}$) as the **length** of the **longest proper suffix** of $\mathbf{pat}[1 \ldots i]$ that **matches a prefix** of $\mathbf{pat}$, such that $\mathbf{pat}[i+1] \neq \mathbf{pat}[\mathbf{SP}_i{+}1]$.

Example

	1	2	3	4	5	6	7	8	9	10	11	12
pat=	b	b	c	c	a	e	b	b	c	a	b	d

$\mathbf{SP}_1 = 0$ $\mathbf{SP}_2 = 1$ $\mathbf{SP}_3 = 0$ $\mathbf{SP}_4 = 0$ $\mathbf{SP}_5 = 0$ $\mathbf{SP}_6 = 0$

$\mathbf{SP}_7 = 0$ $\mathbf{SP}_8 = 1$ $\mathbf{SP}_9 = 3$ $\mathbf{SP}_{10} = 0$ $\mathbf{SP}_{11} = 1$ $\mathbf{SP}_{12} = 0$

Pseudocode to compute $\mathbf{SP}_i$ values for $\mathbf{pat}[1 \ldots m]$ using Z-algorithm (self-study)

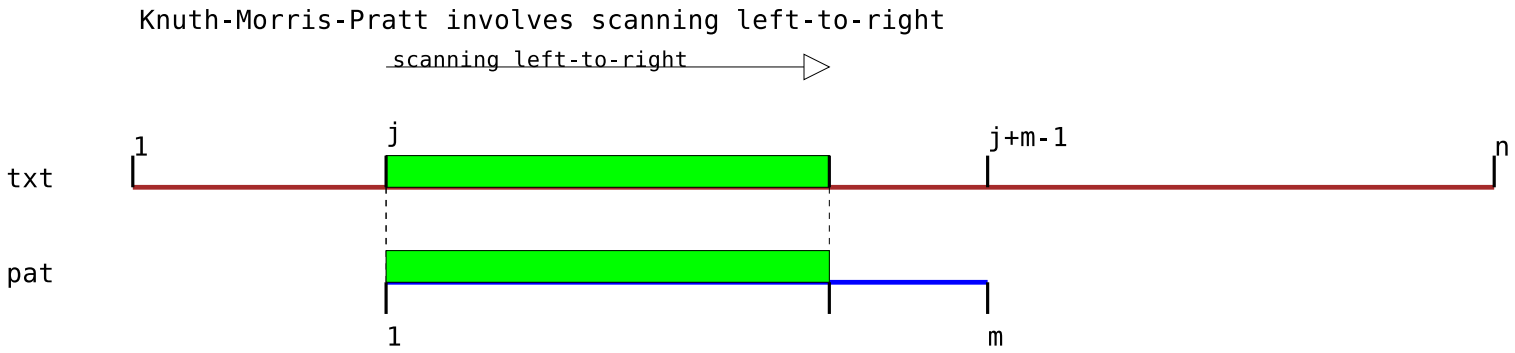
```
Z(1...m) = computeZvalues(pat)
m := |pat|
for i from 1 to m  do
    SP_i := 0
endfor

for j from m down to 2  do
    i := j + Z(j) - 1
    SP_i := Z(j)
endfor
```

KMP Algorithm

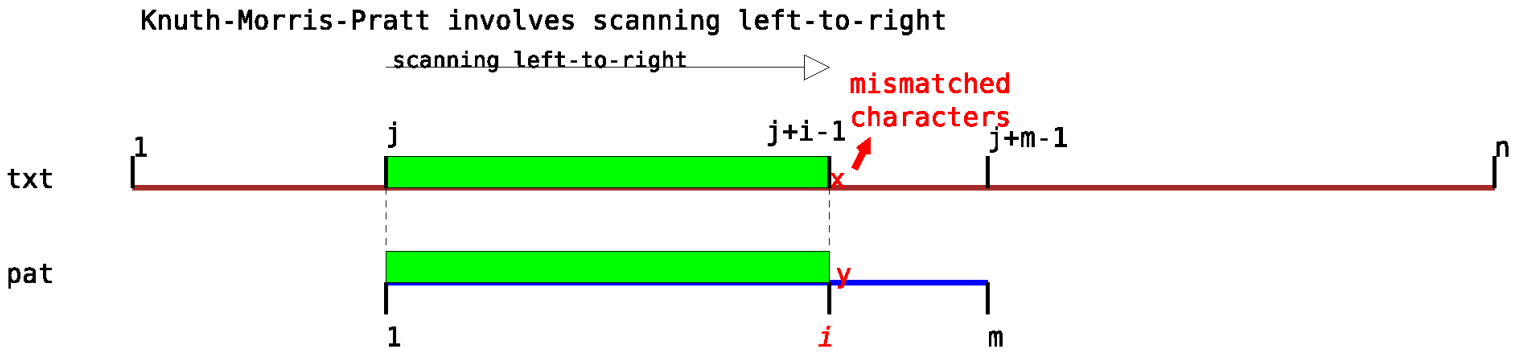
- Preprocess the whole **pat** and identify all $\mathbf{SP}_i$ values.
- The general procedure/iteration of KMP involves:

KMP Algorithm



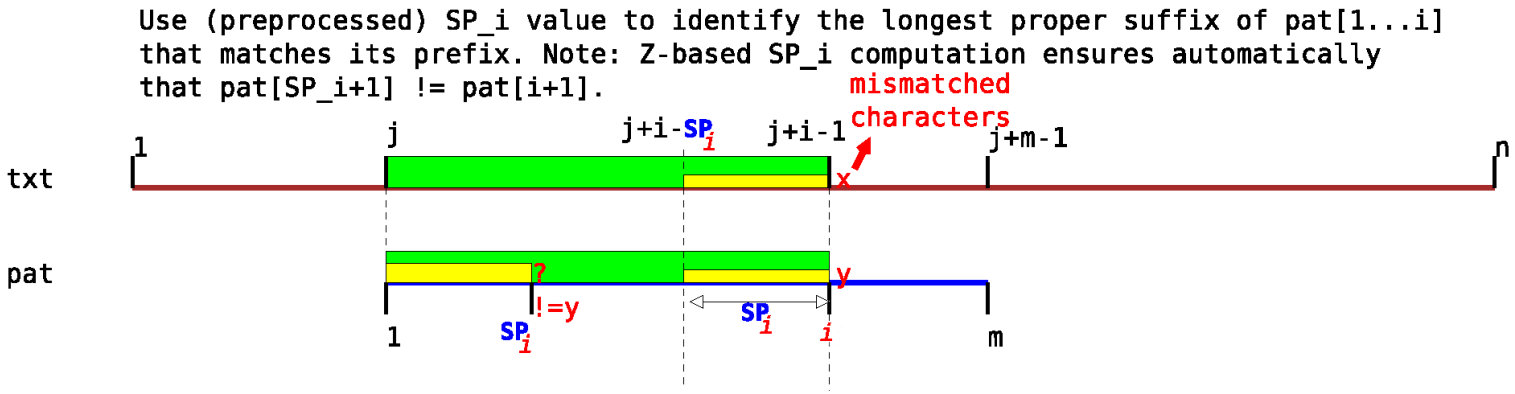
- Preprocess the whole **pat** and identify all $\mathbf{SP}_i$ values.
- The general procedure/iteration of KMP involves:
  - Scanning in the **natural** left-to-right direction the **pat**$[1 \ldots m]$ against a region of **txt**$[j \ldots j + m - 1]$.

KMP Algorithm



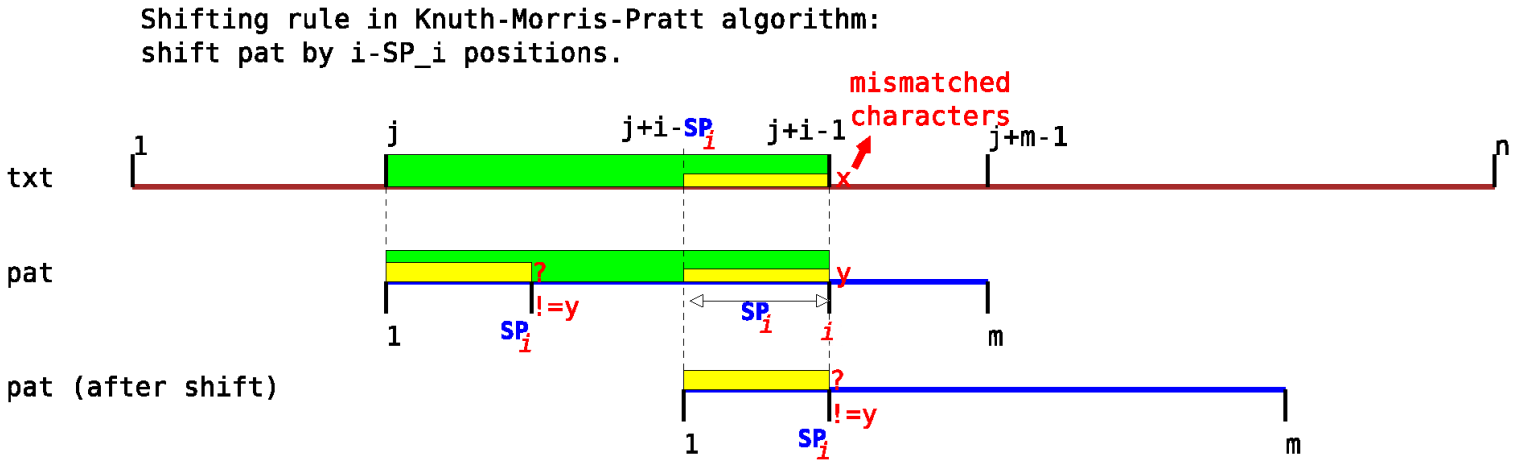
- Preprocess the whole **pat** and identify all $\mathbf{SP}_i$ values.
- The general procedure/iteration of KMP involves:
  - Scanning in the **natural** left-to-right direction the **pat**$[1 \ldots m]$ against a region of **txt**$[j \ldots j+m-1]$.
  - If the first mismatch, scanning left-to-right, occurs at position $i+1$:

KMP Algorithm



- Preprocess the whole **pat** and identify all $\mathbf{SP}_i$ values.
- The general procedure/iteration of KMP involves:
  - Scanning in the **natural** left-to-right direction the **pat**$[1 \ldots m]$ against a region of **txt**$[j \ldots j+m-1]$.
  - If the first mismatch, scanning left-to-right, occurs at position $i+1$:
    - ★ Use precomputed $\mathbf{SP}_i$ to identify the **longest proper suffix** of **pat**$[1 \ldots i]$ that matches its prefix.

KMP Algorithm



- Preprocess the whole **pat** and identify all $\mathbf{SP}_i$ values.
- The general procedure/iteration of KMP involves:
  - Scanning in the **natural** left-to-right direction the **pat**$[1 \ldots m]$ against a region of **txt**$[j \ldots j+m-1]$.
  - If the first mismatch, scanning left-to-right, occurs at position $i+1$:
    - ★ Use precomputed $\mathbf{SP}_i$ to identify the **longest proper suffix** of **pat**$[1 \ldots i]$ that matches its prefix.
    - ★ **KMP shift rule** Shift **pat** rightwards by exactly $i - \mathbf{SP}_i$ places so that **pat**$[1 \ldots \mathbf{SP}_i]$ is now aligned with **txt**$[j+i-\mathbf{SP}_i \ldots j+i-1]$.

KMP Algorithm

- Preprocess the whole **pat** and identify all $\mathbf{SP}_i$ values.
- The general procedure/iteration of KMP involves:
  - Scanning in the **natural** left-to-right direction the **pat**$[1 \ldots m]$ against a region of **txt**$[j \ldots j + m - 1]$.
  - If the first mismatch, scanning left-to-right, occurs at position $i + 1$:
    - ★ Use precomputed $\mathbf{SP}_i$ to identify the **longest proper suffix** of **pat**$[1 \ldots i]$ that matches its prefix.
    - ★ **KMP shift rule** Shift **pat** rightwards by exactly $i - \mathbf{SP}_i$ places so that **pat**$[1 \ldots \mathbf{SP}_i]$ is now aligned with **txt**$[j + i - \mathbf{SP}_i \ldots j + i - 1]$.
  - In the case where an occurrence of **pat** is found in **txt** (i.e., no mismatch), then shift **pat** by $m - \mathbf{SP}_m$ places.

Weeks 1 and 2 summary

- Naïve algorithm takes $\mathcal{O}(mn)$-time, worst case.
- Gusfield's Z algorithm takes $\mathcal{O}(m+n)$-time, worst case.
- Boyer-Moore's algorithm takes
  - $\mathcal{O}(m+n)$-time worst case ...
  - ... but $\mathcal{O}(\frac{n}{m})$-time in most 'realworld' usage.
- Knuth-Morris-Pratt algorithm also takes $\mathcal{O}(m+n)$-time worst-case, but inferior in performance to Boyer-Moore in practice.

Next topic ...

Burrows-Wheeler transform and pattern matching in time proportional to query length.

Preclass learning for Week 3: Revise prefix-doubling suffix array construction learnt in FIT2004.

-=o0o=-

END

-=o0o=-